SYSTEM INFORMATION SYSTEM
HEXAWARE ASSIGNMENT-OOPS

BY-AAYUSHI GUPTA

Task 1: Define Classes
Task 2: Define Constructor
Define the following classes based on the domain description:
Student class with the following attributes:
• Student ID
• First Name
• Last Name
• Date of Birth
• Email
• Phone Number

Code :

```
class Student:
    def __init__(self, student_id, first_name, last_name, date_of_birth, email, phone_number):
        self.student_id = student_id
        self.first_name = first_name
        self.last_name = last_name
        self.date_of_birth = date_of_birth
        self.email = email
        self.phone_number = phone_number
        self.enrollments = []
        self.payments = []

    def Get_student_id(self):
        return self.__student_id

    def Get_first_name(self):
        return self.__first_name

    def Get_last_name(self):
        return self.__last_name

    def Get_date_of_birth(self):
        return self.__date_of_birth
```

```
    def Get_email(self):
        return self.__email

    def Get_phone_number(self):
        return self.__phone_number

    def Set_student_id(self, student_id):
        self.__student_id = student_id

    def Set_first_name(self, first_name):
        self.__first_name = first_name

    def Set_last_name(self, last_name):
        self.__last_name = last_name

    def Set_date_of_birth(self, date_of_birth):
        self.__date_of_birth = date_of_birth

    def Set_email(self, email):
        self.__email = email

    def Set_phone_number(self, phone_number):
        self.__phone_number = phone_number
```

Course class with the following attributes:

- Course ID
- Course Name
- Course Code
- Instructor Name

Code:

```
class Course:
    def __init__(self, course_id, course_name, teacher_id, credits):
        self.course_id = course_id
        self.course_name = course_name
        self.teacher_id = teacher_id
        self.credits = credits

        self.enrollments = []

    def Get_course_id(self):
```

```
        return self.__course_id

    def Get_course_name(self):
        return self.__course_name

    def set_course_id(self, course_id):
        self.__course_id = course_id

    def set_course_name(self, course_name):
        self.__course_name = course_name
```

Enrollment class to represent the relationship between students and courses. It should have attributes:
• Enrollment ID
• Student ID (reference to a Student)
• Course ID (reference to a Course)
• Enrollment Date

Code :

```
class Enrollment:
    def __init__(self,enrollment_id,student_id,course_id,enrollment_date):
        self.__enrollment_id = enrollment_id
        self.student_id = student_id
        self.course_id = course_id
        self.enrollment_date = enrollment_date

    def Get_enrollment_id(self):
        return self.__enrollment_id

    def Get_student_id(self):
        return self.__student_id

    def Get_course_id(self):
        return self.__course_id

    def Get_enrollment_date(self):
        return self.__enrollment_date

    def Set_enrollment_id(self, enrollment_id):
        self.__enrollment_id = enrollment_id
```

```
    def Set_student_id(self, student_id):
        self.__student_id = student_id

    def Set_course_id(self, course_id):
        self.__course_id = course_id

    def Set_enrollment_date(self, enrollment_date):
        self.__enrollment_date = enrollment_date
```

Teacher class with the following attributes:
• Teacher ID
• First Name
• Last Name
• Email

Code:

```
class Teacher:
    def __init__(self, teacher_id, first_name, last_name, email):
        self.teacher_id = teacher_id
        self.first_name = first_name
        self.last_name = last_name
        self.email = email
        self.courses_assigned = []

    def Get_teacher_id(self):
        return self.__teacher_id

    def Get_first_name(self):
        return self.__first_name

    def Get_last_name(self):
        return self.__last_name

    def Get_email(self):
        return self.__email

    def Set_teacher_id(self, teacher_id):
        self.__teacher_id = teacher_id

    def Set_first_name(self, first_name):
```

```
        self.__first_name = first_name

    def Set_last_name(self, last_name):
        self.__last_name = last_name

    def Set_email(self, email):
        self.__email = email
```

Payment class with the following attributes:
• Payment ID
• Student ID (reference to a Student)
• Amount
• Payment Date

Code:

```
class Payment:
    def __init__(self, payment_id, student_id, amount, payment_date):
        self.__payment_id = payment_id
        self.__student_id = student_id
        self.__amount = amount
        self.__payment_date = payment_date

    def Get_payment_id(self):
        return self.__payment_id

    def Get_student_id(self):
        return self.__student_id

    def Get_amount(self):
        return self.__amount

    def Get_payment_date(self):
        return self.__payment_date

    def Set_payment_id(self, payment_id):
        self.__payment_id = payment_id

    def Set_student_id(self, student_id):
        self.__student_id = student_id

    def Set_amount(self, amount):
```

```
        self.__amount = amount

    def Set_payment_date(self, payment_date):
        self.__payment_date = payment_date
```

Task 3: Implement Methods

Student Class:
• EnrollInCourse(course: Course): Enrolls the student in a course.
• UpdateStudentInfo(firstName: string, lastName: string, dateOfBirth: DateTime, email: string,
phoneNumber: string): Updates the student's information.
• MakePayment(amount: decimal, paymentDate: DateTime): Records a payment made by the
student.
• DisplayStudentInfo(): Displays detailed information about the student.
• GetEnrolledCourses(): Retrieves a list of courses in which the student is enrolled.

Code:

```
class StudentServiceImpl(StudentDAO, DBConnection):

    def Add_student(self):
        first_name = input("Enter first name: ")
        last_name = input("Enter last name: ")
        date_of_birth = input("Enter date of birth (YYYY-MM-DD): ")
        email = input("Enter email: ")
        phone_number = input("Enter phone number: ")

        if not all([first_name, last_name, date_of_birth, email, phone_number]):
            raise InvalidStudentDataException("All fields are required.")

        date_of_birth = str(date_of_birth)
        phone_number = str(phone_number)

        self.cursor.execute(
            "INSERT INTO students (first_name, last_name, date_of_birth, email,
             phone_number) VALUES (?, ?, ?, ?, ?)",
            (first_name, last_name, date_of_birth, email, phone_number)
        )
        self.conn.commit()
```

```
        print("Student added successfully!")

    def Update_student(self):
        student_id = input("Enter the student ID to update: ")
        first_name = input("Enter updated first name: ")
        last_name = input("Enter updated last name: ")
        date_of_birth = input("Enter updated date of birth (YYYY-MM-DD): ")
        email = input("Enter updated email: ")
        phone_number = input("Enter updated phone number: ")

        self.cursor.execute(
            "UPDATE students SET first_name=?, last_name=?, date_of_birth=?,
            email=?, phone_number=? WHERE student_id=?",
            (first_name, last_name, date_of_birth, email, phone_number, student_id)
        )
        self.conn.commit()
        print("Student updated successfully!")

    def Get_student(self):
        try:
            student_id = int(input("Enter student ID: "))
            print("Searching for student with ID:", student_id)

        # Execute the query to find the student by ID
            self.cursor.execute("SELECT * FROM students WHERE student_id = ?",
            (student_id,))
            row = self.cursor.fetchone()

            if row:
                print("Student found:", row)
            # Ensure that the row data corresponds to the Student constructor
                student = Student(*row)  # Make sure this is correct according to your
            class definition
                return student
            else:
                raise StudentNotFoundException(f"No student found with ID:
{student_id}")

        except StudentNotFoundException as e:
            print(str(e))  # Handle custom exception if student is not found
        except Exception as e:
```

```
            print("Error retrieving student:", e)

    def Delete_student(self):
        student_id = int(input("Enter student ID: "))

        self.cursor.execute("DELETE FROM students WHERE student_id = ?",
(student_id,))
        self.conn.commit()
        print("Student deleted successfully!")

    def Get_all_students(self):
        try:
            self.cursor.execute("SELECT * FROM students")
            students = [Student(*row) for row in self.cursor.fetchall()]

            if students:
                print("All students:")
                for student in students:
                    print(f"Student ID: {student.student_id}")
                    print(f"First Name: {student.first_name}")
                    print(f"Last Name: {student.last_name}")
                    print(f"Date of Birth: {student.date_of_birth}")
                    print(f"Email: {student.email}")
                    print(f"Phone Number: {student.phone_number}")
                    print()
            else:
                print("No students found.")
        except Exception as e:
            print("Error retrieving students:", e)
```

Course Class:

• AssignTeacher(teacher: Teacher): Assigns a teacher to the course.
• UpdateCourseInfo(courseCode: string, courseName: string, instructor: string): Updates course
information.
• DisplayCourseInfo(): Displays detailed information about the course.
• GetEnrollments(): Retrieves a list of student enrollments for the course.
• GetTeacher(): Retrieves the assigned teacher for the course.

Code:

```
class CourseServiceImpl(CourseDAO, DBConnection):
```

```
    def Add_course(self):
        course_id = int(input("Enter course ID: "))
        course_name = input("Enter course name: ")
        teacher_id = int(input("Enter teacher ID: "))
        credits = int(input("Enter credits: "))

        if not course_name or credits <= 0:
            raise InvalidCourseDataException("Course name is required and credits
should be a positive number.")

        self.cursor.execute(
            "INSERT INTO courses (course_id, course_name, teacher_id, credits)
VALUES (?, ?, ?, ?)",
            (course_id, course_name, teacher_id, credits)
        )
        self.conn.commit()
        print("Course added successfully!")

    def Update_course(self):
        course_id = int(input("Enter course ID: "))
        course_name = input("Enter updated course name: ")
        teacher_id = int(input("Enter updated teacher ID: "))
        credits = int(input("Enter updated credits: "))

        self.cursor.execute(
            "UPDATE courses SET course_name=?, teacher_id=?, credits=?
WHERE course_id=?",
            (course_name, teacher_id, credits, course_id)
        )
        self.conn.commit()
        print("Course updated successfully!")

    def Get_course(self):
        try:
            course_id = int(input("Enter course ID: "))
            self.cursor.execute("SELECT * FROM courses WHERE course_id = ?",
(course_id,))
            row = self.cursor.fetchone()

            if row:
```

```
            course = Course(*row)
            print(f"Course ID: {course.course_id}")
            print(f"Course Name: {course.course_name}")
            print(f"Credits: {course.credits}")
            print(f"Teacher ID: {course.teacher_id}")
        else:
            raise CourseNotFoundException(f"No course found with ID:
{course_id}")

    except CourseNotFoundException as e:
        print(str(e))
    except Exception as e:
        print("Error retrieving course:", e)

  def Delete_course(self):
    course_id = int(input("Enter course ID: "))

    self.cursor.execute("DELETE FROM courses WHERE course_id = ?",
(course_id,))
    self.conn.commit()
    print("Course deleted successfully!")

  def Get_all_courses(self):
    try:
        self.cursor.execute("SELECT * FROM courses")
        courses = [Course(*row) for row in self.cursor.fetchall()]

        if courses:
            for course in courses:
                print(f"Course ID: {course.course_id}")
                print(f"Course Name: {course.course_name}")
                print(f"Credits: {course.credits}")
                print(f"Teacher ID: {course.teacher_id}")
        else:
            print("No courses found.")
    except Exception as e:
        print("Error retrieving courses:", e)
```

Enrollment Class:

- GetStudent(): Retrieves the student associated with the enrollment.
- GetCourse(): Retrieves the course associated with the enrollment.

Code:

```
class EnrollmentServiceImpl(EnrollmentDAO, DBConnection):

    def Add_enrollment(self):
        student_id = int(input("Enter student ID: "))
        course_id = int(input("Enter course ID: "))
        enrollment_date = input("Enter enrollment date: ")

        if not enrollment_date:
            raise InvalidEnrollmentDataException("Enrollment date is required.")

        self.cursor.execute("SELECT * FROM enrollments WHERE student_id = ?
AND course_id = ?", (student_id, course_id))
        existing = self.cursor.fetchone()

        if existing:
            raise DuplicateEnrollmentException("Student is already enrolled in the
course.")

        self.cursor.execute(
            "INSERT INTO enrollments (student_id, course_id, enrollment_date)
VALUES (?, ?, ?)",
            (student_id, course_id, enrollment_date)
        )
        self.conn.commit()
        print("Enrollment added successfully!")

    def Update_enrollment(self):
        enrollment_id = int(input("Enter enrollment ID: "))
        student_id = int(input("Enter updated student ID: "))
        course_id = int(input("Enter updated course ID: "))
        enrollment_date = input("Enter updated enrollment date: ")

        self.cursor.execute(
            "UPDATE enrollments SET student_id=?, course_id=?,
enrollment_date=? WHERE enrollment_id=?",
            (student_id, course_id, enrollment_date, enrollment_id)
        )
        self.conn.commit()
        print("Enrollment updated successfully!")
```

```
    def Get_enrollment(self):
        enrollment_id = int(input("Enter enrollment ID: "))

        self.cursor.execute("SELECT * FROM enrollments WHERE enrollment_id
= ?", (enrollment_id,))
        row = self.cursor.fetchone()

        if row:
            enrollment = Enrollment(*row)
            print(f"Enrollment ID: {enrollment.enrollment_id}")
            print(f"Student ID: {enrollment.student_id}")
            print(f"Course ID: {enrollment.course_id}")
            print(f"Enrollment Date: {enrollment.enrollment_date}")
        else:
            print("No enrollment found with ID:", enrollment_id)

    def Delete_enrollment(self):
        enrollment_id = int(input("Enter enrollment ID: "))

        self.cursor.execute("DELETE FROM enrollments WHERE enrollment_id
= ?", (enrollment_id,))
        self.conn.commit()
        print("Enrollment deleted successfully!")

    def Get_all_enrollments(self):
        try:
            self.cursor.execute("SELECT * FROM enrollments")
            enrollments = [Enrollment(*row) for row in self.cursor.fetchall()]

            if enrollments:
                for enrollment in enrollments:
                    print(f"Enrollment ID: {enrollment.enrollment_id}")
                    print(f"Student ID: {enrollment.student_id}")
                    print(f"Course ID: {enrollment.course_id}")
                    print(f"Enrollment Date: {enrollment.enrollment_date}")
            else:
                print("No enrollments found.")
        except Exception as e:
            print("Error retrieving enrollments:", e)
```

Teacher Class:
• UpdateTeacherInfo(name: string, email: string, expertise: string): Updates teacher information.
• DisplayTeacherInfo(): Displays detailed information about the teacher.
• GetAssignedCourses(): Retrieves a list of courses assigned to the teacher.

Code:

```
class TeacherServiceImpl(TeacherDAO, DBConnection):

    def Add_teacher(self):
        first_name = input("Enter first name: ")
        last_name = input("Enter last name: ")
        email = input("Enter email: ")

        if not all([first_name, last_name, email]):
            raise InvalidTeacherDataException("All fields are required.")

        self.cursor.execute(
            "INSERT INTO teachers (first_name, last_name, email) VALUES
(?, ?, ?)",
            (first_name, last_name, email)
        )
        self.conn.commit()
        print("Teacher added successfully!")

    def Update_teacher(self):
        teacher_id = int(input("Enter teacher ID: "))
        first_name = input("Enter updated first name: ")
        last_name = input("Enter updated last name: ")
        email = input("Enter updated email: ")

        self.cursor.execute(
            "UPDATE teachers SET first_name=?, last_name=?, email=? WHERE
teacher_id=?",
            (first_name, last_name, email, teacher_id)
        )
        self.conn.commit()
        print("Teacher updated successfully!")

    def Get_teacher(self):
        teacher_id = int(input("Enter teacher ID: "))
```

```
        self.cursor.execute("SELECT * FROM teachers WHERE teacher_id = ?",
(teacher_id,))
        row = self.cursor.fetchone()

        if row:
            teacher = Teacher(*row)
            print(f"Teacher ID: {teacher.teacher_id}")
            print(f"First Name: {teacher.first_name}")
            print(f"Last Name: {teacher.last_name}")
            print(f"Email: {teacher.email}")
        else:
            print(f"No teacher found with ID: {teacher_id}")

    def Delete_teacher(self):
        teacher_id = int(input("Enter teacher ID: "))

        self.cursor.execute("DELETE FROM teachers WHERE teacher_id = ?",
(teacher_id,))
        self.conn.commit()
        print("Teacher deleted successfully!")

    def Get_all_teachers(self):
        try:
            self.cursor.execute("SELECT * FROM teachers")
            teachers = [Teacher(*row) for row in self.cursor.fetchall()]

            if teachers:
                for teacher in teachers:
                    print(f"Teacher ID: {teacher.teacher_id}")
                    print(f"First Name: {teacher.first_name}")
                    print(f"Last Name: {teacher.last_name}")
                    print(f"Email: {teacher.email}")
            else:
                print("No teachers found.")
        except Exception as e:
            print("Error retrieving teachers:", e)
```

Payment Class:

- GetStudent(): Retrieves the student associated with the payment.
- GetPaymentAmount(): Retrieves the payment amount.

• GetPaymentDate(): Retrieves the payment date.

Code:

```
class PaymentServiceImpl(PaymentDAO, DBConnection):

   def Add_payment(self):
      enrollment_id = int(input("Enter enrollment ID: "))
      payment_amount = float(input("Enter payment amount: "))

      self.cursor.execute(
         "INSERT INTO payments (enrollment_id, payment_amount) VALUES
(?, ?)",
         (enrollment_id, payment_amount)
      )
      self.conn.commit()
      print("Payment added successfully!")

   def Update_payment(self):
      payment_id = int(input("Enter payment ID: "))
      enrollment_id = int(input("Enter updated enrollment ID: "))
      payment_amount = float(input("Enter updated payment amount: "))

      self.cursor.execute(
         "UPDATE payments SET enrollment_id=?, payment_amount=? WHERE
payment_id=?",
         (enrollment_id, payment_amount, payment_id)
      )
      self.conn.commit()
      print("Payment updated successfully!")

   def Get_payment(self):
      payment_id = int(input("Enter payment ID: "))

      self.cursor.execute("SELECT * FROM payments WHERE payment_id = ?",
(payment_id,))
      row = self.cursor.fetchone()

      if row:
         payment = Payment(*row)
         print(f"Payment ID: {payment.payment_id}")
         print(f"Enrollment ID: {payment.enrollment_id}")
```

```
            print(f"Payment Amount: {payment.payment_amount}")
        else:
            print(f"No payment found with ID: {payment_id}")

    def Delete_payment(self):
        payment_id = int(input("Enter payment ID: "))

        self.cursor.execute("DELETE FROM payments WHERE payment_id = ?",
(payment_id,))
        self.conn.commit()
        print("Payment deleted successfully!")

    def Get_all_payments(self):
        try:
            self.cursor.execute("SELECT * FROM payments")
            payments = [Payment(*row) for row in self.cursor.fetchall()]

            if payments:
                for payment in payments:
                    print(f"Payment ID: {payment.payment_id}")
                    print(f"Enrollment ID: {payment.enrollment_id}")
                    print(f"Payment Amount: {payment.payment_amount}")
            else:
                print("No payments found.")
        except Exception as e:
            print("Error retrieving payments:", e)
```

SIS Class (if you have one to manage interactions):

• EnrollStudentInCourse(student: Student, course: Course): Enrolls a student in a course.

• AssignTeacherToCourse(teacher: Teacher, course: Course): Assigns a teacher to a course.

• RecordPayment(student: Student, amount: decimal, paymentDate: DateTime): Records a payment made by a student.

• GenerateEnrollmentReport(course: Course): Generates a report of students enrolled in a specific course.

• GeneratePaymentReport(student: Student): Generates a report of payments made by a specific
student.
• CalculateCourseStatistics(course: Course): Calculates statistics for a specific course, such as the
number of enrollments and total payments.

Code:

```
class SIS:
    def __init__(self):
        self.students = []
        self.courses = []
        self.teachers = []
        self.enrollments = []
        self.payments = []
```

Task 4: Exceptions handling and Custom Exceptions

Implementing custom exceptions allows you to define and throw exceptions tailored to specific situations or business logic requirements.

• **DuplicateEnrollmentException**: Thrown when a student is already enrolled in a course and tries
to enroll again. This exception can be used in the EnrollStudentInCourse method.
• **CourseNotFoundException**: Thrown when a course does not exist in the system, and you
attempt to perform operations on it (e.g., enrolling a student or assigning a teacher).
• **StudentNotFoundException**: Thrown when a student does not exist in the system, and you
attempt to perform operations on the student (e.g., enrolling in a course, making a payment).
• **TeacherNotFoundException**: Thrown when a teacher does not exist in the system, and you
attempt to assign them to a course.
• **PaymentValidationException**: Thrown when there is an issue with payment validation, such as
an invalid payment amount or payment date.
• **InvalidStudentDataException**: Thrown when data provided for creating or updating a student is
invalid (e.g., invalid date of birth or email format).
• **InvalidCourseDataException**: Thrown when data provided for creating or updating a course is

invalid (e.g., invalid course code or instructor name).
• **InvalidEnrollmentDataException**: Thrown when data provided for creating an enrollment is
invalid (e.g., missing student or course references).
• **InvalidTeacherDataException**: Thrown when data provided for creating or updating a teacher is
invalid (e.g., missing name or email).
• **InsufficientFundsException**: Thrown when a student attempts to enroll in a course but does not
have enough funds to make the payment

Code:

```
class DuplicateEnrollmentException(Exception):
    pass

class CourseNotFoundException(Exception):
    pass

class StudentNotFoundException(Exception):
    pass

class TeacherNotFoundException(Exception):
    pass

class PaymentValidationException(Exception):
    pass

class InvalidDataException(Exception):
    pass

class InvalidStudentDataException(Exception):
    pass

class InvalidCourseDataException(Exception):
    pass

class InvalidEnrollmentDataException(Exception):
    pass

class InvalidTeacherDataException(Exception):
    pass
```

```
class InsufficientFundsException(Exception):
    Pass
```

Task 5: Collections

Student Class:

Create a list or collection property to store the student's enrollments. This property will hold references
to Enrollment objects.
Example: List<Enrollment> Enrollments { get; set; }

Course Class:

Create a list or collection property to store the course's enrollments. This property will hold references
to Enrollment objects.
Example: List<Enrollment> Enrollments { get; set; }

Enrollment Class:

Include properties to hold references to both the Student and Course objects.
Example: Student Student { get; set; } and Course Course { get; set; }

Teacher Class:

Create a list or collection property to store the teacher's assigned courses. This property will hold
references to Course objects.
Example: List<Course> AssignedCourses { get; set; }

Payment Class:

Include a property to hold a reference to the Student object.
Example: Student Student { get; set; }

Code:

```
from abc import ABC, abstractmethod
from typing import List
from Entity.Student import Student
from Entity.Course import Course
from Entity.Enrollment import Enrollment
from Entity.Teacher import Teacher
from Entity.Payment import Payment


class StudentDAO(ABC):
    @abstractmethod
    def Add_student(self):
        pass
```

```
    @abstractmethod
    def Update_student(self):
        pass

    @abstractmethod
    def Get_student(self):
        pass

    @abstractmethod
    def Delete_student(self):
        pass

    @abstractmethod
    def Get_all_students(self) -> List[Student]:
        pass


class CourseDAO(ABC):
    @abstractmethod
    def Add_course(self):
        pass

    @abstractmethod
    def Update_course(self):
        pass

    @abstractmethod
    def Get_course(self):
        pass

    @abstractmethod
    def Delete_course(self):
        pass

    @abstractmethod
    def Get_all_courses(self) -> List[Course]:
        pass


class EnrollmentDAO(ABC):
```

```
    @abstractmethod
    def Add_enrollment(self):
        pass

    @abstractmethod
    def Update_enrollment(self):
        pass

    @abstractmethod
    def Get_enrollment(self):
        pass

    @abstractmethod
    def Delete_enrollment(self):
        pass

    @abstractmethod
    def Get_all_enrollments(self) -> List[Enrollment]:
        pass


class TeacherDAO(ABC):
    @abstractmethod
    def Add_teacher(self):
        pass

    @abstractmethod
    def Update_teacher(self):
        pass

    @abstractmethod
    def Get_teacher(self):
        pass

    @abstractmethod
    def Delete_teacher(self):
        pass

    @abstractmethod
    def Get_all_teachers(self) -> List[Teacher]:
        pass
```

```
class PaymentDAO(ABC):
    @abstractmethod
    def Add_payment(self):
        pass

    @abstractmethod
    def Update_payment(self):
        pass

    @abstractmethod
    def Get_payment(self):
        pass

    @abstractmethod
    def Delete_payment(self):
        pass

    @abstractmethod
    def Get_all_payments(self) -> List[Payment]:
        pass
```

Task 6: Create Methods for Managing Relationships

To add, remove, or retrieve related objects, you should create methods within your SIS class or each relevant class.

• **AddEnrollment**(student, course, enrollmentDate): In the SIS class, create a method that adds an enrollment to both the Student's and Course's enrollment lists. Ensure the Enrollment object references the correct Student and Course.

• **AssignCourseToTeacher**(course, teacher): In the SIS class, create a method to assign a course to a teacher. Add the course to the teacher's AssignedCourses list.

• **AddPayment**(student, amount, paymentDate): In the SIS class, create a method that adds a payment to the Student's payment history. Ensure the Payment object references the correct Student.

• **GetEnrollmentsForStudent**(student): In the SIS class, create a method to retrieve all enrollments
for a specific student.
• **GetCoursesForTeacher**(teacher): In the SIS class, create a method to retrieve all courses
assigned to a specific teacher.

Code:

```
class Student:
    def __init__(self, student_id, first_name, last_name):
        self.student_id = student_id
        self.first_name = first_name
        self.last_name = last_name
        self.enrollments = []
        self.payments = []

class Course:
    def __init__(self, course_id, course_name):
        self.course_id = course_id
        self.course_name = course_name
        self.enrollments = []

class Teacher:
    def __init__(self, teacher_id, first_name, last_name):
        self.teacher_id = teacher_id
        self.first_name = first_name
        self.last_name = last_name
        self.assigned_courses = []

class Enrollment:
    def __init__(self, enrollment_id, student, course, enrollment_date):
        self.enrollment_id = enrollment_id
        self.student = student
        self.course = course
        self.enrollment_date = enrollment_date

class Payment:
    def __init__(self, payment_id, student, amount, payment_date):
        self.payment_id = payment_id
        self.student = student
        self.amount = amount
```

```
        self.payment_date = payment_date

class SIS:
    def __init__(self):
        self.students = []
        self.courses = []
        self.teachers = []

    # Task 1: Add Enrollment
    def add_enrollment(self, student, course, enrollment_date):
        enrollment = Enrollment(len(student.enrollments) + 1, student, course,
enrollment_date)
        student.enrollments.append(enrollment)
        course.enrollments.append(enrollment)
        print(f"Enrollment added: {student.first_name} {student.last_name} ->
{course.course_name}")

    # Task 2: Assign Course to Teacher
    def assign_course_to_teacher(self, course, teacher):
        teacher.assigned_courses.append(course)
        print(f"Course '{course.course_name}' assigned to {teacher.first_name}
{teacher.last_name}")

    # Task 3: Add Payment
    def add_payment(self, student, amount, payment_date):
        payment = Payment(len(student.payments) + 1, student, amount,
payment_date)
        student.payments.append(payment)
        print(f"Payment of {amount} added for {student.first_name}
{student.last_name} on {payment_date}")

    # Task 4: Get Enrollments for a Student
    def get_enrollments_for_student(self, student):
        return [f"{e.course.course_name} (Enrolled on: {e.enrollment_date})" for e
in student.enrollments]

    # Task 5: Get Courses for a Teacher
    def get_courses_for_teacher(self, teacher):
        return [course.course_name for course in teacher.assigned_courses]

# Example usage:
```

```
sis = SIS()

# Create sample students, courses, and teachers
student1 = Student(1, "John", "Doe")
course1 = Course(101, "Introduction to Programming")
course2 = Course(102, "Mathematics 101")
teacher1 = Teacher(201, "Sarah", "Smith")

# Add students, courses, and teachers to the SIS
sis.students.append(student1)
sis.courses.extend([course1, course2])
sis.teachers.append(teacher1)

# Task 1: Enroll John Doe in courses
sis.add_enrollment(student1, course1, "2023-09-01")
sis.add_enrollment(student1, course2, "2023-09-02")

# Task 2: Assign Sarah Smith to a course
sis.assign_course_to_teacher(course1, teacher1)

# Task 3: Add a payment for John Doe
sis.add_payment(student1, 500.00, "2023-10-01")

# Task 4: Retrieve enrollments for John Doe
enrollments = sis.get_enrollments_for_student(student1)
print(f"Enrollments for {student1.first_name} {student1.last_name}: {',
'.join(enrollments)}")

# Task 5: Retrieve courses for Sarah Smith
courses = sis.get_courses_for_teacher(teacher1)
print(f"Courses assigned to {teacher1.first_name} {teacher1.last_name}: {',
'.join(courses)}")
```

```
Enrollment added: John Doe -> Introduction to Programming
Enrollment added: John Doe -> Mathematics 101
Course 'Introduction to Programming' assigned to Sarah Smith
Payment of 500.0 added for John Doe on 2023-10-01
Enrollments for John Doe: Introduction to Programming (Enrolled on: 2023-09-01), Mathemat
Courses assigned to Sarah Smith: Introduction to Programming
```

Task 7: Database Connectivity

Code:

```
import pyodbc
from util.dbconnection import PropertyUtil


# When new object -> new connection
class DBConnection:
    def __init__(self):
        conn_str = PropertyUtil.get_property_string()
        self.conn = pyodbc.connect(conn_str)
        self.cursor = self.conn.cursor()

    def close(self):
        self.cursor.close()
        self.conn.close()

import pyodbc


class PropertyUtil:
    @staticmethod
    def get_property_string():
        server_name = "LAPTOP-N5SA57O6\MSSQLSERVER01"
        database_name = "SISDataBase"

        conn_str = (
            f"Driver={{SQL Server}};"
            f"Server={LAPTOP-N5SA57O6\MSSQLSERVER01};"
            f"Database={SISDataBase};"
            f"Trusted_Connection=yes;"
        )

        return conn_str
```

Task 8: Student Enrollment

In this task, a new student, John Doe, is enrolling in the SIS. The system needs to record John's information, including his personal details, and enroll him in a few courses. Database connectivity is required to store this information.

John Doe's details:
• First Name: John
• Last Name: Doe
• Date of Birth: 1995-08-15
• Email: john.doe@example.com
• Phone Number: 123-456-7890
John is enrolling in the following courses:
• Course 1: Introduction to Programming
• Course 2: Mathematics 101
The system should perform the following tasks:
• Create a new student record in the database.
• Enroll John in the specified courses by creating enrollment records in the database.

Code:

```
import pyodbc

# Establish database connection
def get_db_connection():
    conn = pyodbc.connect(
        'DRIVER={SQL Server};'
        'SERVER=LAPTOP-N5SA57O6\MSSQLSERVER01;'
        'DATABASE=SISDataBase;'
        'Trusted_Connection=yes;'
    )
    return conn

# Task 1: Create a new student record in the database
def create_student(first_name, last_name, dob, email, phone):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('''
        INSERT INTO Students (first_name, last_name, date_of_birth, email,
phone)
        VALUES (?, ?, ?, ?, ?)
    ''', (first_name, last_name, dob, email, phone))
    conn.commit()

    # Retrieve the student ID for the newly created student
    cursor.execute('SELECT @@IDENTITY AS student_id')
    student_id = cursor.fetchone()[0]
```

```
    conn.close()
    return student_id

# Task 2: Enroll the student in specified courses
def enroll_student(student_id, course_names):
    conn = get_db_connection()
    cursor = conn.cursor()

    for course_name in course_names:
        cursor.execute('''
            SELECT course_id FROM Courses WHERE course_name = ?
        ''', (course_name,))
        course_id = cursor.fetchone()[0]

        cursor.execute('''
            INSERT INTO Enrollments (student_id, course_id, enrollment_date)
            VALUES (?, ?, GETDATE())
        ''', (student_id, course_id))

    conn.commit()
    conn.close()

# Main function to enroll John Doe in two courses
def enroll_john_doe():
    # John's personal details
    first_name = 'John'
    last_name = 'Doe'
    dob = '1995-08-15'
    email = 'john.doe@example.com'
    phone = '123-456-7890'

    # Enroll in these two courses
    courses = ['Introduction to Programming', 'Mathematics 101']

    # Create a student record and retrieve student ID
    student_id = create_student(first_name, last_name, dob, email, phone)

    # Enroll John Doe in the specified courses
    enroll_student(student_id, courses)
    print(f"John Doe (Student ID: {student_id}) enrolled in courses: {',
'.join(courses)}")
```

```
# Execute the enrollment for John Doe
enroll_john_doe()
```

OUTPUT:

```
John Doe (Student ID: 103) enrolled in courses: Introduction to Programming, Mathematics 101
```

Task 9: Teacher Assignment

In this task, a new teacher, Sarah Smith, is assigned to teach a course. The system needs to update the course record to reflect the teacher assignment.

Teacher's Details:

- Name: Sarah Smith
- Email: sarah.smith@example.com
- Expertise: Computer Science

Course to be assigned:

- Course Name: Advanced Database Management
- Course Code: CS302

The system should perform the following tasks:

- Retrieve the course record from the database based on the course code.

Code:

```
def get_course_by_code(course_code):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('''
        SELECT course_id, course_name, instructor_name
        FROM courses
        WHERE course_code = ?
    ''', (course_code,))
    course = cursor.fetchone()
    conn.close()
    return course if course else "Course not found."
```

- Assign Sarah Smith as the instructor for the course.

Code:

```
def assign_instructor(course_code, instructor_name, email, expertise):
    # Assuming the teacher's information needs to be stored first if not already in the database
    conn = get_db_connection()
```

```
    cursor = conn.cursor()

    # Insert the teacher into the teacher's table if not exists (optional)
    cursor.execute('''
        INSERT INTO teachers (first_name, last_name, email, expertise)
        VALUES (?, ?, ?, ?)
    ''', (instructor_name.split()[0], instructor_name.split()[1], email, expertise))

    conn.commit()

    # Retrieve teacher ID for linking with the course
    cursor.execute('''
        SELECT teacher_id
        FROM teachers
        WHERE email = ?
    ''', (email,))
    teacher_id = cursor.fetchone()[0]

    # Now assign the instructor to the course
    cursor.execute('''
        UPDATE courses
        SET instructor_name = ?, teacher_id = ?
        WHERE course_code = ?
    ''', (instructor_name, teacher_id, course_code))

    conn.commit()
    conn.close()
    return "Instructor assigned successfully."
```

• Update the course record in the database with the new instructor information.

Code:

```
def assign_sarah_to_course():
    course_code = "CS302"
    instructor_name = "Sarah Smith"
    email = "sarah.smith@example.com"
    expertise = "Computer Science"

    # Task 1: Retrieve the course record
    course = get_course_by_code(course_code)
```

```
    print(course)

    # Task 2: Assign the instructor to the course
    print(assign_instructor(course_code, instructor_name, email, expertise))
```

```
PS C:\Users\asus\OneDrive\Desktop\Assignment_2_Hexaware-main> python -m main.Mainmodule
Welcome to  the Student Information System

 (302, 'Advanced Database Management', None)

 Instructor assigned successfully.
```

Task 10: Payment Record

In this task, a student, Jane Johnson, makes a payment for her enrolled courses. The system needs to record this payment in the database.

Jane Johnson's details:

- Student ID: 101
- Payment Amount: $500.00
- Payment Date: 2023-04-10

The system should perform the following tasks:

- Retrieve Jane Johnson's student record from the database based on her student ID.

Code:

```
def get_student_record(student_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('''
        SELECT student_id, first_name, last_name, outstanding_balance
        FROM students
        WHERE student_id = ?
    ''', (student_id,))
    student = cursor.fetchone()
    conn.close()
    return student if student else "Student not found."
```

- Record the payment information in the database, associating it with Jane's student record.

Code:

```
def record_payment(student_id, amount, payment_date):
    conn = get_db_connection()
```

```
    cursor = conn.cursor()
    cursor.execute('''
        INSERT INTO payments (student_id, amount, payment_date)
        VALUES (?, ?, ?)
    ''', (student_id, amount, payment_date))
    conn.commit()
    conn.close()
    return "Payment recorded successfully."
```

• Update Jane's outstanding balance in the database based on the payment amount.

Code:

```
def update_outstanding_balance(student_id, payment_amount):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('''
        UPDATE students
        SET outstanding_balance = outstanding_balance - ?
        WHERE student_id = ?
    ''', (payment_amount, student_id))
    conn.commit()
    conn.close()
    return "Outstanding balance updated successfully."
```

```
 def process_jane_payment():
    student_id = 101
    payment_amount = 500.00
    payment_date = "2023-04-10"

    # Task 1: Retrieve Student Record
    student = get_student_record(student_id)
    print(student)

    # Task 2: Record Payment
    print(record_payment(student_id, payment_amount, payment_date))

    # Task 3: Update Outstanding Balance
    print(update_outstanding_balance(student_id, payment_amount))
```

```
PS C:\Users\asus\OneDrive\Desktop\Assignment_2_Hexaware-main> python -m main.Mainmodule
Welcome to  the Student Information System
```

```
(101, 'Jane', 'Johnson', 800.00)
Payment recorded successfully.
Outstanding balance updated successfully.
```

Task 11: Enrollment Report Generation

In this task, an administrator requests an enrollment report for a specific course, "Computer Science 101." The system needs to retrieve enrollment information from the database and generate a report.

Course to generate the report for:

• Course Name: Computer Science 101

The system should perform the following tasks:

• Retrieve enrollment records from the database for the specified course.

Code:

```
def get_enrollment_by_course(course_name):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('''
        SELECT s.student_id, s.first_name, s.last_name, s.email, e.enrollment_date
        FROM enrollments e
        JOIN students s ON e.student_id = s.student_id
        JOIN courses c ON e.course_id = c.course_id
        WHERE c.course_name = ?
    ''', (course_name,))
    enrollments = cursor.fetchall()
    conn.close()
    return enrollments
```

```
Retrieving the course record for course code CS302...
Course found: (302, 'Advanced Database Management', None)
Assigning Sarah Smith as the instructor...
Instructor assigned successfully.
```

• Generate an enrollment report listing all students enrolled in Computer Science 101.

Code:

```
def generate_enrollment_report(course_name):
    enrollments = get_enrollment_by_course(course_name)
```

```
        if not enrollments:
        return "No students are enrolled in this course."

    report = f"Enrollment Report for {course_name}\n"
    report += "=" * 40 + "\n"
    report += "{:<10} {:<15} {:<15} {:<30} {:<15}\n".format("Student ID",
"First Name", "Last Name", "Email", "Enrollment Date")
    report += "-" * 40 + "\n"

    for enrollment in enrollments:
        report += "{:<10} {:<15} {:<15} {:<30} {:<15}\n".format(
            enrollment[0], enrollment[1], enrollment[2], enrollment[3], enrollment[4]
        )

    return report
```

```
Retrieving student record for Student ID: 101...
Student found: Jane Johnson
Recording payment of $500.00 made on 2023-04-10...
Payment recorded successfully.
Updating Jane Johnson's outstanding balance...
Outstanding balance updated successfully.
```

• Display or save the report for the administrator.

Code:

```
def display_enrollment_report(course_name):
    report = generate_enrollment_report(course_name)
    print(report)  # To display the report in the console

    # Optionally save it to a file (text)
    with open(f"{course_name}_enrollment_report.txt", "w") as file:
        file.write(report)
    print(f"Report saved as {course_name}_enrollment_report.txt")
```

Output:

```
Retrieving enrollment records for Computer Science 101...
Enrollment Report for Computer Science 101
=========================================
Student ID First Name      Last Name       Email                          Enrollment Date
-----------------------------------------
101        John            Doe             john.doe@example.com           2023-09-01
102        Jane            Johnson         jane.johnson@example.com       2023-09-02
Report saved as Computer Science 101_enrollment_report.txt
```